Proiectarea și implementarea aplicației mySSH

Alexa Daniel-Andrei[0009−0007−5365−6061]

Facultatea de Informatică Iași, Universitatea "Alexandru Ioan Cuza"
daniel-andrei.alexa@info.uaic.ro

Abstract. Aplicația mySSH reproduce funcționalitatea protocolului Secure Shell, fiind bazată pe o arhitectură client-server și implementată în limbajul C folosind standardul POSIX. Soluția se bazează pe utilizarea funcției forkpty pentru emularea unui terminal virtual pentru fiecare client, ceea ce permite execuția aplicațiilor interactive (precum editoarele de text), gestionarea culorilor, utilizarea comenzilor înlănțuite cât și a redirectărilor. Securitatea comunicației este asigurată prin criptarea traficului TCP folosind biblioteca OpenSSL, prevenind interceptarea datelor.

Keywords: Secure Shell · OpenSSL · Forkpty · POSIX · C

1 Introducere

Acest proiect constă în dezvoltarea unei aplicații de tip Secure Shell, cu o arhitectură client-server. Scopul principal este facilitarea unui mod prin care un utilizator se poate conecta la un server de la distanță pentru a executa comenzi shell (Bash), cât și pentru a putea încărca fișiere pe server. Aplicația oferă un terminal virtual funcțional, cu suport pentru culori și editoare interactive (ex. vim, nano), ce poate fi redimensionat, datele fiind criptate pe durata comunicării dintre client și server. De asemenea, activitatea utilizatorilor este reținută în fișierul server.log, împreună cu momentul în care aceasta a fost realizată.

2 Tehnologii Aplicate

Pentru implementare, se utilizează următoarele tehnologii:

TCP/IP: Protocolul de transport utilizat este TCP, deoarece conexiunea shell necesită transportul garantat al tuturor pachetelor (acestea trebuie să ajungă în ordine și fără erori).

OpenSSL: Utilizat pentru criptarea datelor comunicate prin TCP/IP. Luând în considerare faptul că se transmit informații private (parole, comenzi de sistem), transportul datelor în clar este vulnerabil acțiunilor de tip "packet sniffing", fapt ce ar compromite aceste informații.

Forkpty și Termios: Utilizate pentru emularea terminalului. Forkpty permite serverului să creeze un pseudo-terminal (PTY) care permite procesului Bash să creadă că este conectat la un monitor real, iar termios este folosit pentru client pentru a configura terminalul in modul raw (input-urile de la tastatură sunt trimise instant ca raw bytes, sunt ignorate semnalele generate de combinații precum CTRL+C, CTRL+Z etc).

3 Structura Aplicației

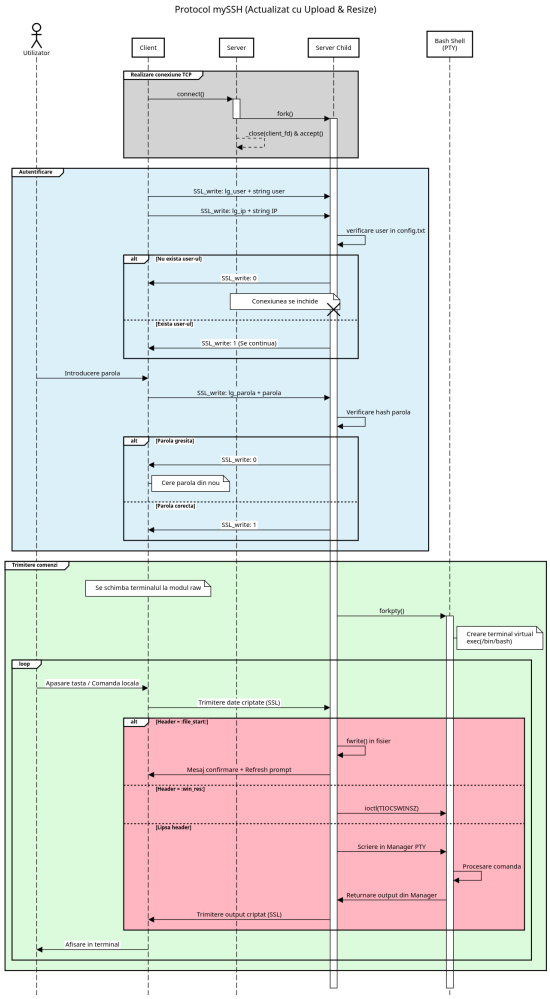

Fig. 1. Diagrama aplicației

Aplicația are o arhitectură concurentă bazată pe procese.

Serverul: Ascultă conexiuni pe portul 6097. La acceptarea unui client, se realizează un fork iar procesul copil se ocupă de handshake-ul SSL și autentificare. După acestea, se realizează un al doilea fork folosind forkpty, ce creează un proces nepot, în care va rula bash, astfel procesul copil rămânând intermediar între conexiunea criptată SSL și terminalul virtual.

Clientul: Inițializează conexiunea SSL, realizează autentificarea, apoi configurează terminalul local în modul raw. Primește input de la tastatură pe care îl criptează și decriptează output-ul primit de la server pentru a-l afișa.

4 Protocolul de Comunicare

Inițial se realizează un fork, pentru a avea un proces separat asociat fiecărui client. Ulterior, protocolul este împărțit în două etape: autentificarea și trimiterea comenzilor.

4.1 A. Etapa de autentificare

Client ⟶ Server: 4 bytes lungime user
Server ⟵ Client: 4 bytes lungime user
Client ⟶ Server: string user
Server ⟵ Client: string user
Client ⟶ Server: 4 bytes lungime IP
Server ⟵ Client: 4 bytes lungime IP
Client ⟶ Server: string IP
Server ⟵ Client: string IP
Server ⟶ Client:
"0" dacă nu se găsește user-ul primit de la client in fișierul config.json
"1" dacă se găsește user-ul primit de la client în fișierul config.json
Client ⟵ Server:
dacă citește "0" se închide procesul
dacă citește "1" se transmite parola user-ului
Client ⟶ Server: 4 bytes lungime parolă
Server ⟵ Client: 4 bytes lungime parolă
Client ⟶ Server: string parolă
Server ⟵ Client: string parolă
Server ⟶ Client:
"0" dacă parola primită nu corespunde cu cea din fișierul config.json
"1" dacă parola primită corespunde cu cea din fișierul config.json
Client ⟵ Server:
dacă citește "0" se retrimite parola până este introdusă cea corectă sau este întrerupt procesul clientului
dacă citește "1" se continuă la trimiterea comenzilor

4.2 B. Etapa de trimitere a comenzilor

Odată autentificat, dacă se primește header-ul ":win_res:", atunci terminalul virtual primește noile dimensiuni. Dacă se primește combinația CTRL+B de la client, se trimite header-ul ":file_start:", și se transferă datele fișierului precizat (care trebuie să se afle în același director unde este și clientul) la server (în acel director în care se află clientul la acel moment). Altfel se transmite un flux continuu de bytes criptați prin SSL. Orice byte scris în client este trimis imediat la server și ulterior la bash iar orice byte generat de terminalul virtual este trimis imediat la server iar apoi la client.

5 Scenarii de Utilizare

5.1 Succes:

- **Autentificare și listare fișiere:** Utilizatorul "george" se conectează și introduce parola corectă. Primește promptul colorat obișnuit unui terminal. Execută ls -la și vede lista fișierelor completă.
- **Editare fișier:** Utilizatorul deschide fișierul test.txt folosind comanda nano. Interfața se redesenează. Acesta scrie text, salvează schimbările și închide editorul. Aplicația răspunde corect la comenzile speciale și informația persistă in fișierul modificat.
- **Comandă lungă:** Utilizatorul rulează o comandă care generează mult text (ex: yes "exemplu"). Datele sunt transmise continuu între terminalul virtual și client, fără blocaje, datorită modului non-blocant pentru I/O (O_NONBLOCK).

5.2 Eșec:

- **Utilizator invalid:** Clientul trimite un user care nu există în config.json. Serverul răspunde cu "0", clientul afișează eroare și închide conexiunea.
- **Parolă greșită:** Utilizatorul există, dar parola este greșită. Se reîncearcă introducerea parolei până la introducere celei corecte sau până la întreruperea procesului client.
- **Server indisponibil:** Clientul încearcă conectarea, dar serverul este oprit. Clientul primește eroare la connect() și se închide procesul client.

6 Concluzii

Soluția descrisă oferă o modalitate de accesare a unui server, de la distanță, pe care pot fi executate diverse instrucțiuni, datele fiind securizate pe durata comunicării prin TCP/IP. Utilizarea forkpty permite folosirea unor comenzi interactive precum nano, vim, apt install etc, cât și încărcarea fișierelor pe server. Câteva funcționalități ce ar putea fi adăugate în viitor sunt: descărcarea datelor de pe server și autentificarea pe bază de chei publice RSA, ce ar elimina necesitatea parolelor.

References

1. Tanenbaum, A.S., Feamster, N., Wetherall, D.: Computer Networks. 6th edn. Pearson (2021)
2. Ylonen, T., Lonvick, C.: The Secure Shell (SSH) Protocol Architecture. RFC 4251, IETF (2006).
3. Gamble, D.: cJSON: Ultralightweight JSON parser in ANSI C. GitHub Repository, https://github.com/DaveGamble/cJSON (2009).
4. The OpenSSL Project: OpenSSL: The Open Source toolkit for SSL/TLS. https://www.openssl.org/ (2024).